

1)

```
saanvi_kompelli@DESKTOP-2U40RG7:~$ nano VirtualAddress.c
saanvi_kompelli@DESKTOP-2U40RG7:~$ gcc VirtualAddress.c
saanvi_kompelli@DESKTOP-2U40RG7:~$ gcc VirtualAddress.c -o VirtualAddress
saanvi_kompelli@DESKTOP-2U40RG7:~$ ./VirtualAddress
Value of x= 10
Virtual adress of x=0x7ffeaea78834
```

 saanvi_kompelli@DESKTOP-2U40RG7: ~

GNU nano 8.7.1

```
#include <stdio.h>
int main(){
int x=10;
printf("Value of x= %d\n", x);
printf("Virtual adress of x=%p\n", (void *)&x);
return 0;
}
```

2)

```
saanvi_kompelli@DESKTOP-2U40RG7:~$ nano Address.c
saanvi_kompelli@DESKTOP-2U40RG7:~$ gcc Address.c
saanvi_kompelli@DESKTOP-2U40RG7:~$ gcc Address.c -o Address
saanvi_kompelli@DESKTOP-2U40RG7:~$ ./Address
Code address : 0x5d7ad79aa1a9
Data address : 0x5d7ad79ad010
BSS address : 0x5d7ad79ad018
Heap address : 0x5d7af8e90010
Stack address : 0x7ffecf0595bc
```

 saanvi_kompelli@DESKTOP-2U40RG7: ~

GNU nano 8.7.1

```
#include <stdio.h>
#include <stdlib.h>
int global = 10;
int bss_variable;
int main(){
int local=20;
int *heap=malloc(sizeof(int));
*heap=30;

printf("Code address : %p\n", (void *)main);
printf("Data address : %p\n", (void *)&global);
printf("BSS address : %p\n", (void *)&bss_variable);
printf("Heap address : %p\n", (void *)heap);
printf("Stack address : %p\n", (void *)&local);

free(heap);

return 0;
}
```

3)

```
saanvi_kompelli@DESKTOP-2U40RG7:~$ nano Demandpaging.c
saanvi_kompelli@DESKTOP-2U40RG7:~$ gcc Demandpaging.c
saanvi_kompelli@DESKTOP-2U40RG7:~$ gcc Demandpaging.c -o Demandpaging
saanvi_kompelli@DESKTOP-2U40RG7:~$ ./Demandpaging
Enter page number (0-3): 200
Page is already in memory.
Physical frame = 1000
```

 saanvi_kompelli@DESKTOP-2U40RG7: ~

GNU nano 8.7.1

```
#include <stdio.h>

int main() {

    int page_table[4] = {1, -1, 3, -1};

    int page;

    printf("Enter page number (0-3): ");
    scanf("%d", &page);

    if (page_table[page] == -1) {
        printf("Page Fault!\n");
        printf("Page must be loaded into memory.\n");
    }
    else {
        printf("Page is already in memory.\n");
        printf("Physical frame = %d\n", page_table[page]);
    }

    return 0;
}
```

4)

```
saanvi_kompelli@DESKTOP-2U40RG7:~$ nano Adresstranslation.c
saanvi_kompelli@DESKTOP-2U40RG7:~$ gcc Adresstranslation.c
saanvi_kompelli@DESKTOP-2U40RG7:~$ gcc Adresstranslation.c
saanvi_kompelli@DESKTOP-2U40RG7:~$ gcc Adresstranslation.c -o Adresstranslation
saanvi_kompelli@DESKTOP-2U40RG7:~$ ./Adresstranslation
Virtual Address = 250
Page Number      = 2
Offset           = 50
Frame Number     = 1
Physical Address = 150
```

 saanvi_kompelli@DESKTOP-2U40RG7: ~

GNU nano 8.7.1

```
#include <stdio.h>
```

```
int main() {
```

```
    int page_table[] = {3, 5, 1, 7};
```

```
    int page_size = 100;
```

```
    int virtual_address = 250;
```

```
    int page = virtual_address / page_size;
```

```
    int offset = virtual_address % page_size;
```

```
    int frame = page_table[page];
```

```
    int physical_address = frame * page_size + offset;
```

```
    printf("Virtual Address = %d\n", virtual_address);
```

```
    printf("Page Number      = %d\n", page);
```

```
    printf("Offset           = %d\n", offset);
```

```
    printf("Frame Number     = %d\n", frame);
```

```
    printf("Physical Address = %d\n", physical_address);
```

```
    return 0;
```

```
}
```

5)

```
saanvi_kompelli@DESKTOP-2U40RG7:~$ nano Memoryallocation.c
saanvi_kompelli@DESKTOP-2U40RG7:~$ gcc Memoryallocation.c
saanvi_kompelli@DESKTOP-2U40RG7:~$ gcc Memoryallocation.c -o Memoryallocation
saanvi_kompelli@DESKTOP-2U40RG7:~$ ./Memoryallocation
10 20 30 40 50
```

 saanvi_kompelli@DESKTOP-2U40RG7: ~

```
GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>

int main() {

    int *p;

    p = malloc(5 * sizeof(int));

    if (p == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        p[i] = (i + 1) * 10;
    }

    for (int i = 0; i < 5; i++) {
        printf("%d ", p[i]);
    }

    printf("\n");

    free(p);

    return 0;
}
```

6)
6)

```
saanvi_kompelli@DESKTOP-2U40RG7:~$ nano Memoryallocation.c
saanvi_kompelli@DESKTOP-2U40RG7:~$ nano Segmentationfault.c
saanvi_kompelli@DESKTOP-2U40RG7:~$ gcc Segmentationfault.c
saanvi_kompelli@DESKTOP-2U40RG7:~$ gcc Segmentationfault.c -o Segmentationfault
saanvi_kompelli@DESKTOP-2U40RG7:~$ ./Segmentationfault
Before accessing memory
Segmentation fault                  ./Segmentationfault
```

 saanvi_kompelli@DESKTOP-2U40RG7: ~

GNU nano 8.7.1

```
#include <stdio.h>

int main() {

    int *p = NULL;

    printf("Before accessing memory\n");

    *p = 100;

    printf("After accessing memory\n");

    return 0;
}
```